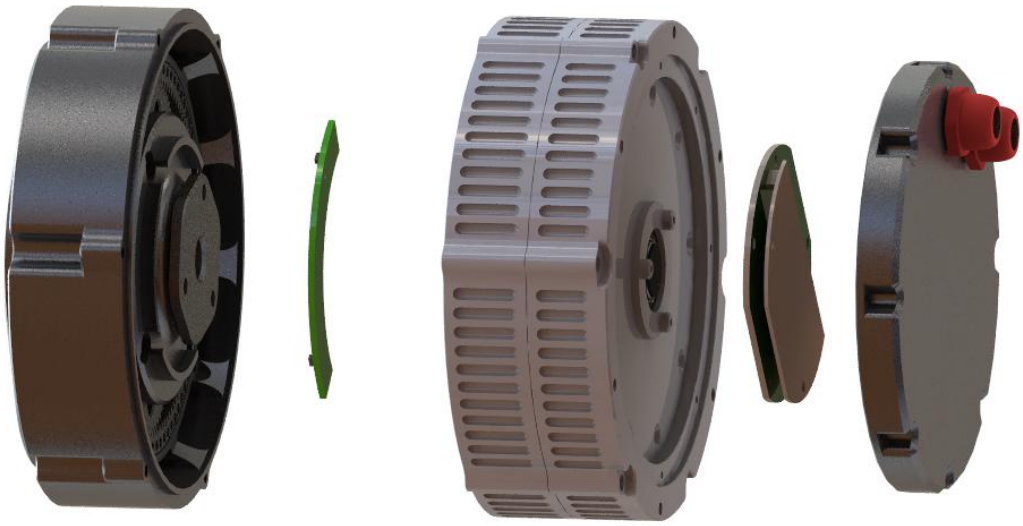


# SEmotor-C 集成驱动器使用说明

# 目录

**一、产品简介.....1**

1.1 概述.....1

1.2 电气参数.....1

**二、上位机使用说明.....2**

2.1 上位机的连接.....2

2.2 其他功能说明.....3

**三、 电机驱动板通信.....5**

3.1 CAN 通讯.....5

3.2 Ethercat 通讯.....7

# 一、产品简介

## 1.1 概述

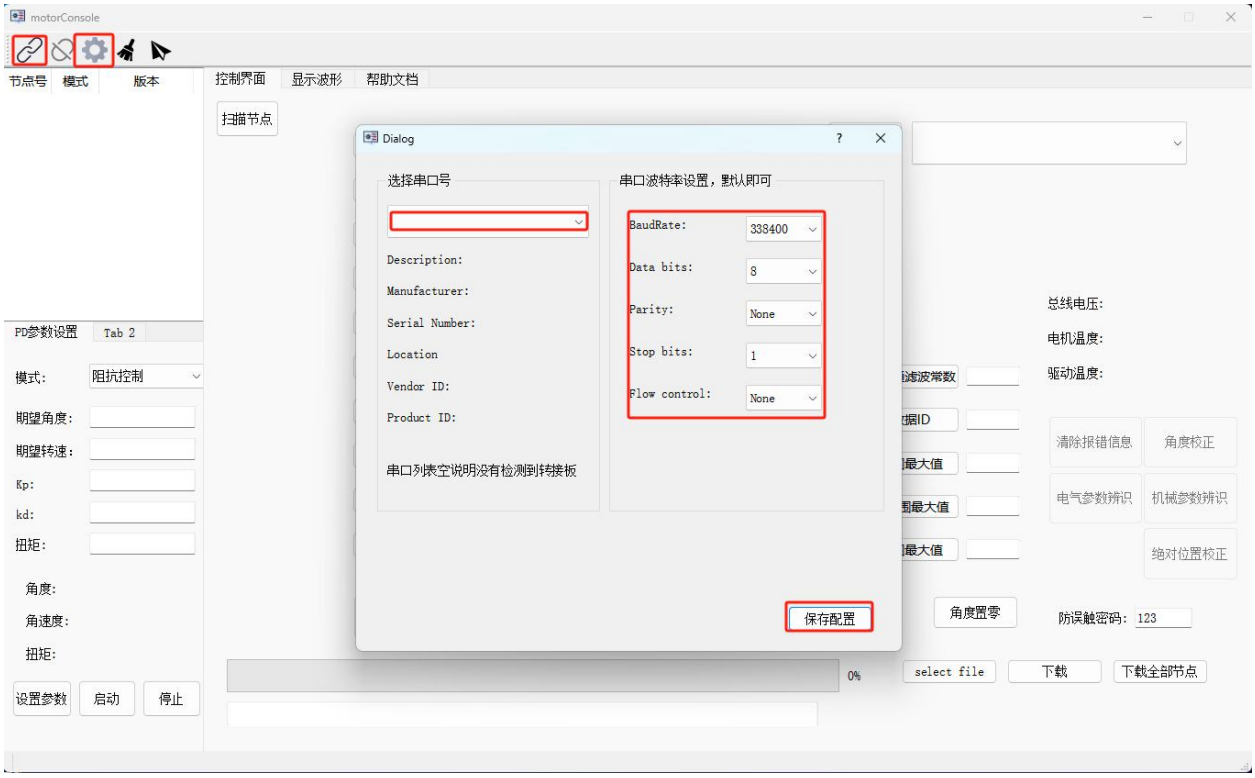
SEmotor-C 集成驱动器是一款集阻抗模式（力矩模式）、速度环、位置环三种控制模式于一体的高精度双编伺服控制器。具备多层保护机制，包括电机、驱动温度保护，电机过流保护，软件限位保护，断线失能保护等。其通讯方式为自定义 CAN 和简易 Ethercat 两种，均提供例程 SDK（ethercat 提供基于 SOEM 主站库及测试例程）。搭配我司轴向磁场关节电机，帮助客户实现快速、简便、稳定的实物部署。

## 1.2 电气参数

| 类型 | 单位 | 参数 |
|---|---|---|
| 工作电压 | V | 16-54.6（85） |
| 最大相电流 | Amp | 70（140） |
| 额定相电流 | Amp | 14（26） |
| 最高转速 | rad/s | 14 |
| 绝对值编码器精度 | 位 | 14 |
| 角度范围 | rad | ±3.14 |
| 最低工作环境温度 | ℃ | -20 |
| 最高工作环境温度 | ℃ | 45 |
| 电流采样频率 | Hz | 20K |

# 二、上位机使用说明

## 2.1 上位机的连接

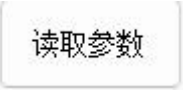



（1）通过随货配送的 CAN 转 USB 调试板，将电机 CAN 线与调试版连接，连接调试板与主机设备。

（2）安装好 CH340 串口驱动。打开 Windows 系统下的 motorConsole 软件，首先点击齿轮状通讯设置图标打开 dialog 界面，选择正确的 COM 口，确认波特率为默认值 338400。设备首次使用上位机需要点击右下角保存设置。

（3）点击锁链图标 ，连接配置板。此时链接图标会变灰色不可操作状态，同时断开链接图标 会变亮。

（4）点击扫描节点，根据节点号选择需要控制的电机(节点号为 CAN ID，默认都为 1，首次使用需依次连接单个电机更改 CAN ID，否则会 ID 冲突导致扫描节点异常)。

（5）连接成功后左下角电机实时信息更新，角度、速度、扭矩、总线电压、电机温度、驱动温度实时变化。点击中间的读取参数 

，将驱动器内部参数读到上位机中。此时可对 CAN ID、Ethercat ID、软件限位开关和限位角度、温度保护阈值、最大输出电流进行设置（电机参数为出厂标定值，请勿擅自更改，其他参数更改步骤为读取参数-更改参数值-设置全部-将参数写入 flash）。此时即可通过左下角运控指令框发送合适的参数和对电机进行控制。

## 2.2 其他功能说明

### (1)固件升级功能

如我司有新版本驱动固件发布，可通过上位机固件升级功能进行更新。通过 select file 选择固件版本，选择正确节点号后下载即可。select file 下载 下载全部节点

（下载空文件或错误文件会导致内部固件丢失，请在公司技术人员指导下进行，切勿擅自点击下载按钮）

### (2)回包参数设置

返回数据ID _____ 设置返回数据的帧 ID，默认都为 0 处于最高优先级。通过 角度范围最大值 _____ 角速度范围最大值 _____ 扭矩范围最大值 _____ 设置电机控制指令数值范围，（需同步例程头文件更改）。角度置零 以当前角度为绝对 0 位（偏置值储存于驱动器 flash 内）。

### (3)错误信息

错误信息 点击错误信息可读取当前报错，点开箭头浏览全部报错，详情对照 fault list 表格。

## (4) 示波器

上位机内带有简易示波器，勾选需要的参数即可描绘波形，采样频率较低（约 10Hz，采集数据越多，频率越低）仅为控制参考用。

显示波形使用方法:

1, ctrl + 鼠标滚轮调节 y 轴范围

2. 单独鼠标滚轮调节 X 轴范围

3. 点击鼠标左键显示 x, y 坐标，按住鼠标左键拖动坐标轴。

4. 拖动坐标轴会取消跟踪最新数据，按右键跟踪，跟踪时也可以调整坐标轴。

# 三、电机驱动板通信

## 3.1 CAN 通讯

电机驱动可通过 CAN 控制电机，设置电机参数。CAN 的波特率为 1Mbts，发送给电机的数据帧为 8 个字节共 64 位，电机返回的数据帧为 6 个字节共 48 位。

CAN 数据协议的指令有：特殊指令，参数指令，控制指令。

### 特殊指令表如下：

表 1 CAN 特殊指令表

| Command | 模式或功能 | 备注 |
|---|---|---|
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFC | MOTOR_MODE | 电机使能 |
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFD | REST_MODE | 电机失能 |
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFE | ZERO_MODE | 角度置零 |
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xF9 | 修改电机 ID( 1-15) | 输出此指令时 CAN 网络仅可连接单个电机，否则会导致 CAN 网络中电机 id 紊乱 |
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xF7 | 角度矫正 | 转子自转一周，自动修正磁编码器偏差 |
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xF6 | 绝对位置矫正 | 电机自传一周，自动修正绝对位置偏差 |
| 0x80 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xF4 | 清除电机错位标志 | 清除电机报错信息 |

### 参数指令

参数指令是特殊指令表中的一种特殊情况，主要用于配置电机参数，读取电机内部数据，Data[0]~ Data[8]为 CAN 指令的 8 个数据位。

| Data[0] | Data[1] | Data[2] | Data[3] | Data[4] | Data[5] | Data[6] | Data[7] |
|---|---|---|---|---|---|---|---|
| 0x80 | value | | | | wirte/read | data_type | 0xEC |

value 为配置参数的值，读取参数时配置为 0。value 为 float 类型，内联 uint8_t[4]数据类型的数组。以小端模式发送——即 Data[1]存放 value 的低位，Data[4]存放 value 的高位。Data[5]为 1 时表示写入，为 0 表示读取。Data[6]表示读写哪个参数，具体含义查看示例例程。

## 控制指令

当第一个字节的最高位为 0（即 data[0]&0x80 == 0）时， 电机控制期望数据设置。控制指令分为阻抗控制，速度控制，位置控制。根据参数指令中的类型

```
#define MOTOR_WR_CONTROL_MODE  0x5B   //0-阻抗控制模式 1-速度控制模式 2-位置控制模式
```

可配置电机的运行哪种控制方式。具体通信协议参考例程。

**阻抗控制：**

在控制指令中，：将期望的电机位置、速度、扭矩，Kp，Kd 等发送给电机。

参数在电机内部的处理公式为:

角度单位为弧度 rad，角速度单位为弧度每秒 rad/s，扭矩单位为牛米 Nm

$$torque_ref = kp * (p_des - angle) + kd * (v_des - velocity) + t_ff$$

$torque_ref$为期望扭矩，$kp$为上图所示 KP 参数，$p_des$为上图所示期望角度，$angle$为电机实际角度，$kd$为上图所示 KD 参数，$v_des$为上图所示期望角速度，$velocity$为电机实际角速度，$t_ff$为前馈扭矩值

**速度控制**

$$torque_ref = V_{kp} * (v_des - velocity) + V_{ki} * Dt * \Sigma(v_des - velocity)$$

$torque_ref$为期望扭矩，$V_{kp}$为速度环比例项系数，$v_des$为上图所示期望角速度，$velocity$为电机实际角速度，$V_{ki}$为速度环积分项系数，Dt 为控制周期 50us。

**位置控制**

$$v_des = kp * (p_des - angle) + kd * (0 - velocity)$$

$$torque_ref = V_{kp} * (v_des - velocity) + V_{ki} * Dt * \Sigma(v_des - velocity)$$

$kp$为位置环比例系数，$p_des$为上图所示期望角度，$angle$为电机实际角度，$kd$为位置环微分系数，$torque_ref$为期望扭矩，$V_{kp}$为速度环比例项系数，$v_des$为上图所示期望角速度，$velocity$为电机实际角速度，$V_{ki}$为速度环积分项系数，Dt 为控制周期 50us。

## 接收的电机数据

当电机接收到指令时返回数据，此过程约 1us。返回的 CAN 数据帧为 6 个 uint8_t 类型的数据，默认所有电机返回 CAN 的 ID 帧为 0，根据数据帧中 data[0]的 ID 判断返回的数据属于哪个电机。具体的解码过程见例程。其中 data[0]包含的电机信息如下：

| Bit7 | Bit6 | Bit5 | Bit4 | Bit3 | Bit2 | Bit1 | Bit0 |
|---|---|---|---|---|---|---|---|
| ACK | FAULT | * | ENABLE | CAN_ID | | | |

| Bit | Field | Type | Default | Description |
|---|---|---|---|---|
| 7 | ACK | R | 0b | 电机接收到指令置 1，接收到数据置 0 |
| 6 | FAULT | R | 0b | 电机驱动出现驱动错误时置 1，该位为 1 时，应立即停止所有电机动作。 |
| 5 | * | R | 0b | 保留 |
| 4 | ENABLE | R | 0b | 电机使能时置 1，失能置 0 |
| 3 | CAN_ID | R | 0b | 电机 ID 标志位，范围 0-15 |
| 2 | | | | |
| 1 | | | | |
| 0 | | | | |

# 3.2 Ethercat 通讯

**运行系统:Ubuntu 20.04.6 LTS。**

## 简述

**根目录文件夹中/lib 中 libsoem.a，为 Ethercat 主站基于 SOEM 的静态库。将 libsoem.a 与头文件 rt_ethercat.hpp 添加至您的工程，实现简便的 Ethercat 通信。控制对象有 cnt（物理连接顺序）和 Ecat_id(上位机设置)两种，详细控制指令见 rt_ethercat.hpp。**

## 测试例程

修改 main 程序中网卡名和电机数量和运行周期（1-2KHz）。

```
Ecat_motor motor("enp60s0", 1000, 1);//网卡名,线程运行周期(单位 us),电机数量
```

例程修改并 cmake 编译后运行可执行文件，先通过“a”或“s”选择控制对象（a 对应 cnt 控制对象 ID，s 对应 Ecat_id 控制对象 ID）；随后输入对应的控制指令 case，再发送使能指令，即可控制电机运行。（控制指令与使能指令皆有两种控制对象区别，详情见 **rt_ethercat.hpp**）

例如：

```
case 's': std::cout<<"input number\n";                    //输入电机序号，选择控制其他电机
                std::cin>>cnt;
                if(cnt < 1 || cnt > motor._motor_num){
                  cnt = 1;
                  std::cout<<"error input,now cnt is 1\n";
                }
        break;
case 'a': std::cout<<"input Ecat_idr\n";                 //输入电机序号，选择控制其他电机
                std::cin>>Ecat_id;
                if(Ecat_id < 1 ){
                  Ecat_id = 1;
                  std::cout<<"error input,now cnt is 1\n";
              }
```

Ecat_id 指令使能、失能

```
motor.Id_control_order(Ecat_id,DISABLE_MODE);            //Ecat_id 控制失能电机
motor.Id_control_order(Ecat_id,ENABLE_MODE);             //Ecat_id 控制使能电机
```

cnt 指令使能、失能

```
motor.control_order(cnt,DISABLE_MODE);                   //cnt 控制失能电机
motor.control_order(cnt,ENABLE_MODE);                    //cnt 控制使能电机
```

Ecat_id 指令阻抗控制

```
case 'k':         Icmomond[cnt -1].KP = 0;                    //Ecat_id 阻抗控制 5Nm
                  Icmomond[cnt -1].KD = 0;
                  Icmomond[cnt -1].velocity = 0;
                  Icmomond[cnt -1].torque  = 5;
                  motor.Id_command(Ecat_id,&Icmomond[cnt -1]); //设置对于 Ecat_id 的控制指令
                  break;
```

cnt 指令阻抗控制

```
case 'm':         Icmomond[cnt -1].torque  = 5;               //cnt 阻抗控制 5Nm
                  Icmomond[cnt -1].KD  = 0;
                  Icmomond[cnt -1].KP  = 0;
                  Icmomond[cnt -1].velocity = 0;
                  Icmomond[cnt -1].angle = 0;
                  motor.command(cnt,&Icmomond[cnt -1]);       //设置对于 cnt 的电机控制指令
                  break;
```